

Operating Systems

Chapters 4, 6 & 7

Threads, Synchronization & Classical Problems

Silberschatz, Galvin & Gagne, 10th Edition

Chapter 4: Threads & Concurrency

Motivation: Most modern applications are multithreaded. Process creation is heavy-weight; thread creation is light-weight. Threads can simplify code and increase efficiency.

Threads vs Processes: A multithreaded process has threads that share code section, data section, heap, open files — but each thread has its own registers, program counter, and stack.

Benefits: Responsiveness, Resource sharing, Economy, Scalability.

Multicore Challenges: Dividing activities, Balance, Data splitting, Data dependency, Testing/debugging.

Concurrency vs Parallelism: Concurrency = more than one task making progress (can be single core). Parallelism = performing more than one task simultaneously (multiple cores).

Multithreading Models: Many-to-One (many user → one kernel), One-to-One (each user → own kernel), Many-to-Many (many user → many kernel), Two-level (many-to-many with binding).

Thread Libraries: POSIX Pthreads, Windows threads, Java threads.

Java Threads: Implement Runnable interface or extend Thread class. Java Executor Framework separates task creation from execution, enabling thread pools for efficient resource management.

Chapter 6: Synchronization Tools

Race Conditions: Concurrent access to shared data without synchronization causes inconsistency.

Critical-Section Problem: When one process in CS, no other may be. Three requirements: (1) Mutual Exclusion, (2) Progress, (3) Bounded Waiting.

Solutions: Peterson's Solution works on single-core but breaks on multicore due to instruction reordering. Fix: use memory barriers.

Hardware Instructions: `test_and_set` (returns original, sets true atomically), `compare_and_swap` (swaps only if matches expected).

Mutex Locks: Boolean lock requiring `acquire()` before CS, `release()` after. Results in spinlock (busy waiting).

Semaphores: Integer accessed via `wait()` and `signal()`. Counting semaphore (any value), Binary semaphore (0 or 1). No-busy-wait version uses waiting queues with `block()/wakeup()`.

Monitors: High-level abstraction with automatic mutual exclusion. Condition variables: `wait()` (suspend), `signal()` (resume one).

Liveness Issues: Deadlock (waiting for impossible events), Starvation (indefinite blocking), Priority Inversion (solved via priority inheritance).

Chapter 7: Synchronization Examples

Classical Problems: Bounded-Buffer, Readers-Writers, Dining-Philosophers — benchmark problems for testing synchronization schemes.

Bounded-Buffer: Producer adds items; consumer removes. Three semaphores: mutex, empty, full. Order critical: wait(empty) before wait(mutex) prevents deadlock.

Readers-Writers: Multiple readers simultaneously; only one writer (with no readers). First variation: readers favored → writer starvation. Second variation: writers favored → reader starvation.

Dining-Philosophers: N philosophers alternate thinking/eating, each needs 2 chopsticks. Naive solution causes deadlock (circular wait). Monitor solution prevents deadlock but not starvation.

Windows Synchronization: Uniprocessor (interrupt masks), Multiprocessor (spinlocks), Dispatcher objects (mutexes, semaphores, events, timers).

Java Synchronization: Monitors (synchronized keyword), wait()/notify(), Reentrant Locks (lock/unlock with try/finally), Semaphores (acquire/release), Condition Variables (await/signal).

Alternative Approaches: Transactional Memory (atomic blocks execute without explicit locks), Functional Programming (immutable variables eliminate races — Erlang, Scala).